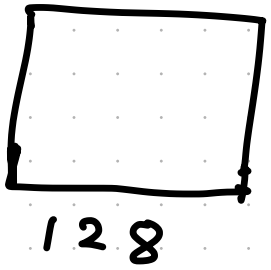
# Image Classification



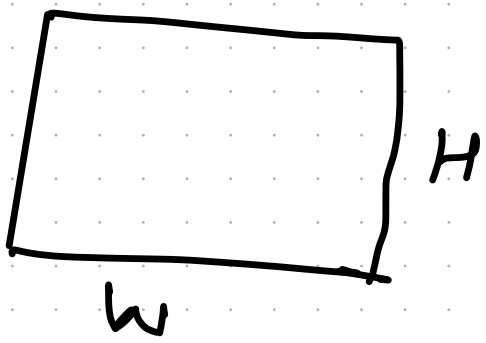

R, G, B : Total values: $3 \times 2^{14}$

- Create a flat vector of $3 \times 2^{14}$ values: $\bar{x}$
- Five way classification
- W : $5 \times (3 \times 2^{14})$
- Prediction vector: $W \times \bar{x}$ : $5 \times 1$
- Softmax over the prediction vector

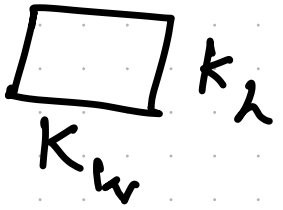
# Problems with this approach

- Ignores the 2D nature of image data
- Linear classifier

# Convolution Operator over 2D data




Data




Kernel

bias (scaler)

- Divide data into overlapping tiles of size $K_h \times K_w$
- Example $W = 128$ $H = 64$ $K_w = 4$, $K_h = 6$, bias
- Elementwise product + bias
- Result: $(58+1) \times (124+1)$

$H'$ $W'$

# Convolution for common image data

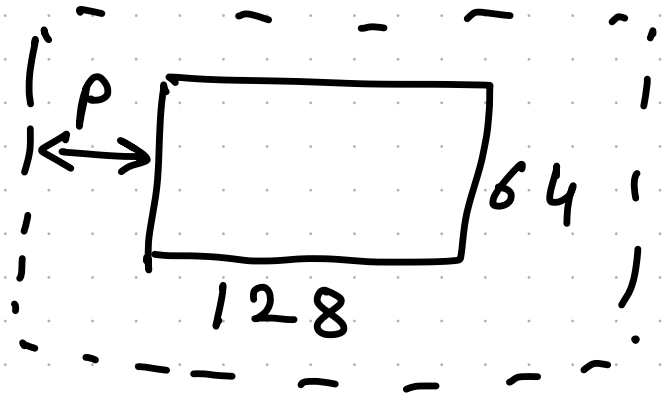
- Input: $C_{in} \times H \times W$, $C_{in}$ = Number of channels
- Generally $C_{in} = 3$ (R, G, B)
  Can vary for special types of images
- Filter: $C_{in} \times K_h \times K_w$ + bias
- If multiple filters: $C_{out} \times C_{in} \times K_h \times K_w$
  + $C_{out}$ biases
- Output: $C_{out} \times H' \times W'$

$H' = (H - K_h) + 1$ $\quad W' = (W - K_w) + 1$

# Example of One convolution layer

- $3 \times 64 \times 128$ Image
- 5 filters of size $4 \times 4$ + 5 biases
- 5 matrices of size $61 \times 125$
  - $61$ ↑ Height
  - $125$ ↑ Width
- Treat this as image of $61 \times 125$ resolution with 5 channels
- Now we can apply convolution recursively
- Problem solved: Utilize the 2D nature of data
- Problem created: Reducing size of representation

# Padding helps to maintain size



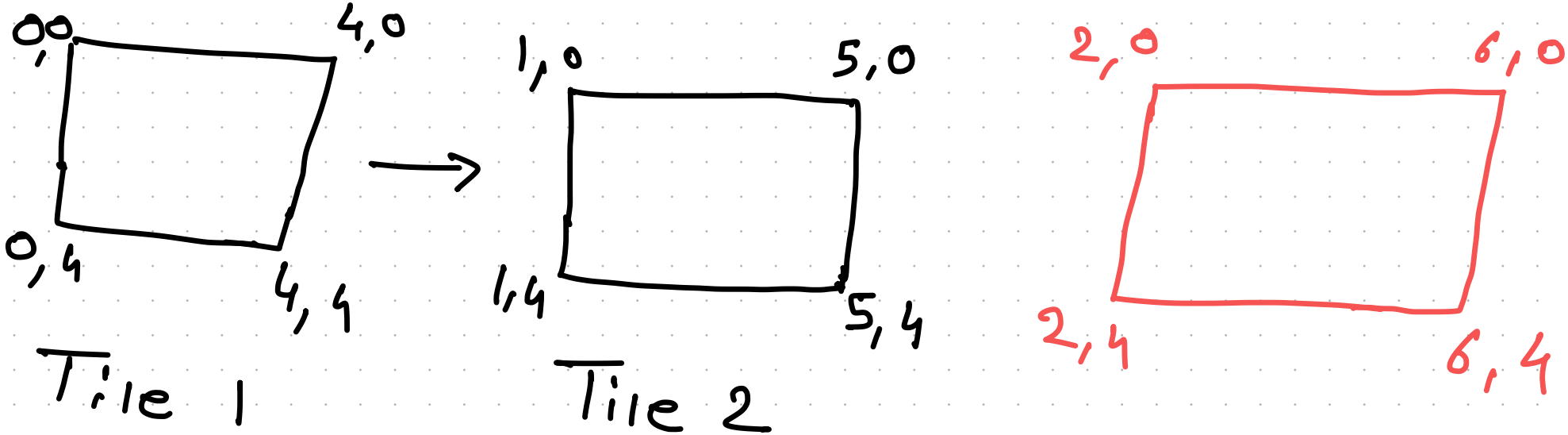

- Add padding of size p around image with all zeros.

$$H' = H - K_h + 2p + 1$$

$$W' = W - K_w + 2p + 1$$

- Generally $K_h = K_w$
- To maintain size: $K = 2p + 1$

# Stride helps to reduce overlapping



Larger Stride scales down the representation

$$H' = \frac{(H - K_h + 2P)}{S} + 1$$

$$W' = \frac{(W - K_w + 2P)}{S} + 1$$

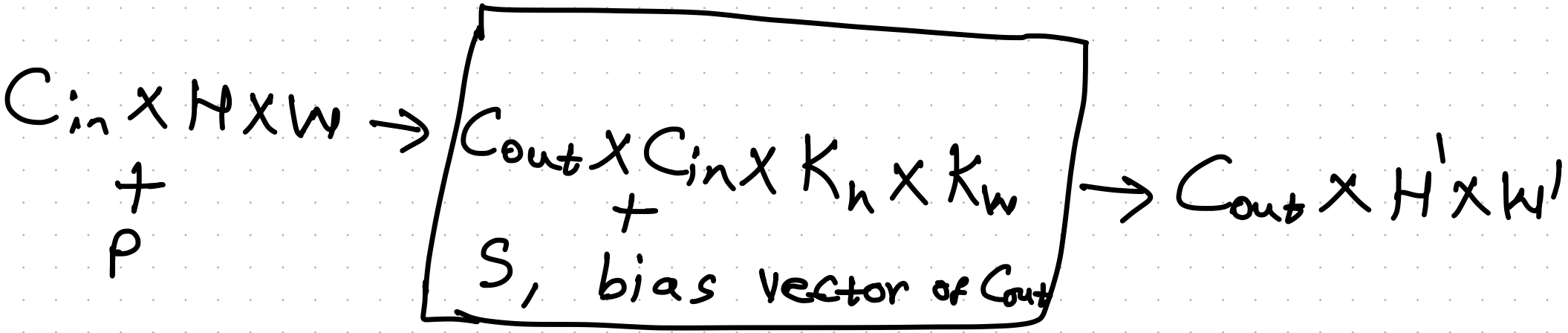
Now we have five knobs

$C_{out}$ : Control the channels in representation

P : Increase or maintain resolution

S : Scale down the representation

$K_h$ and $K_w$ : Create summary of a tile



# A sequence of convolutions is still linear

- Introduce non-linearity
  - Activation
  - Pooling

- Activation: Each element of output goes through an activation function

# Pooling summarizes a tile with a scaler

- Average: Linear
- Min, Max: Non linear
- Pooling window of 2x2 reduces size to $1/4$.
- Each channel is still preserved
- Pooling can also have overlapping tiles
- $C_{in} \times H \times W \rightarrow$ [Pooling] $\rightarrow C_{in} \times H' \times W'$

  $H' < H \quad W' < W$

- Global Average Pooling (GAP): Reduce representation to the vector of size $C_{in}$

Inference is performed on final vector

- GAP
- Flattening
- Fully connected layer or more complex classifier

Convolution can be performed on any data

3D data

$C_{in} \times H \times W \times D$ : Input

$C_{out} \times C_{in} \times K_h \times K_w \times K_d$ : Conv layer

1D data

$C_{in} \times L$ : Input

$C_{out} \times C_{in} \times K$ : Conv layer

## We are working on interesting problems

- Faithful compression of DL models
- Reasoning-as-a-Service
- Fine-grained Multi-LLM collaboration
- Data efficient domain adaptation of LLMs
- Efficient alternatives for Transformers

Join us if you are interested

aweKar@iitg.ac.in

fac.iitg.ac.in/aweKar